# Configuring a Network ACL (NACL)

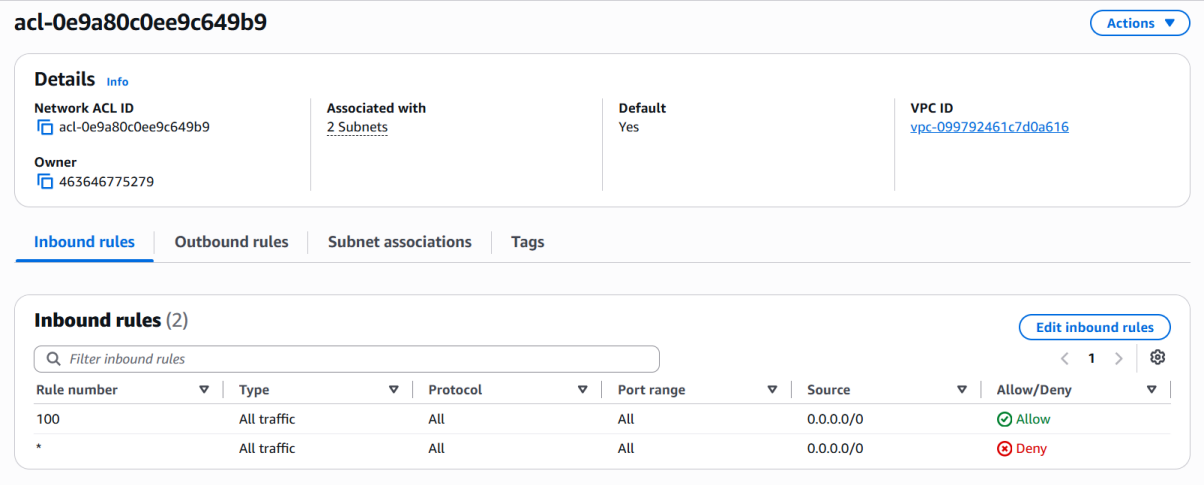
# To Begin with the Lab

### Summary of the Lab

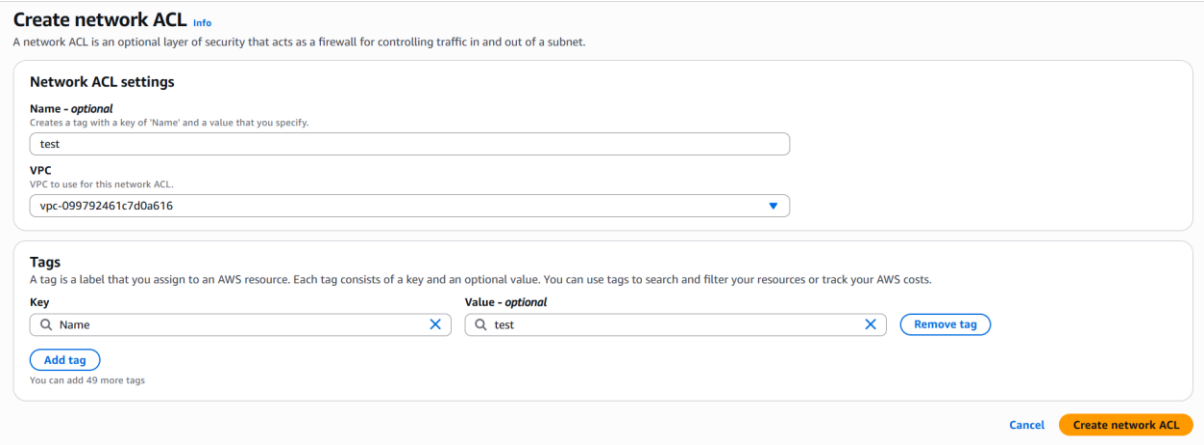
**Note:** This lab assumes that a VPC already exists with two EC2 instances running in the same subnet. Both instances use a Security Group that allows **All Traffic**, so the focus of this lab is on understanding and configuring **Network ACLs (NACLs)**.

Open the AWS Management Console and navigate to the VPC service. Identify the subnet containing the two EC2 instances and verify that both instances are accessible through SSH, Ping, and HTTP before applying any Network ACL rules.

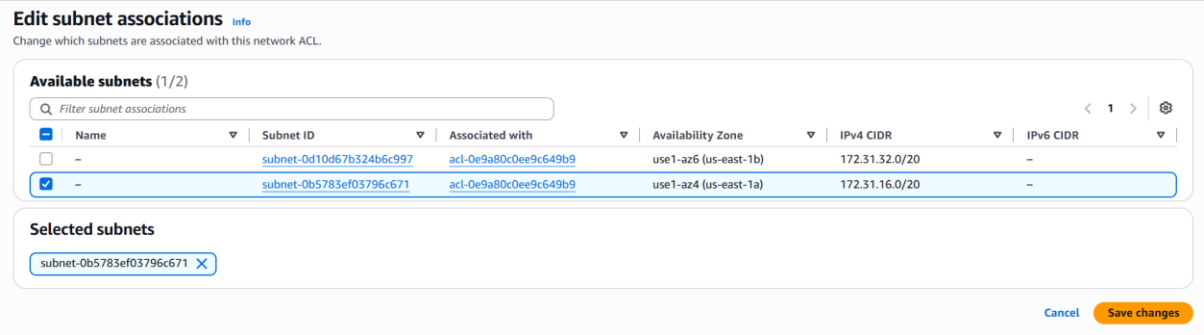
Open the Network ACLs section and observe that the subnet is currently associated with the Default Network ACL. By default, the Default Network ACL allows all inbound and outbound traffic, which is why the EC2 instances are fully accessible.




Click **Create Network ACL**, provide a name such as **Test**, select the existing VPC, and create the Network ACL. At this stage, the new Network ACL is not associated with any subnet and therefore does not affect any resources.

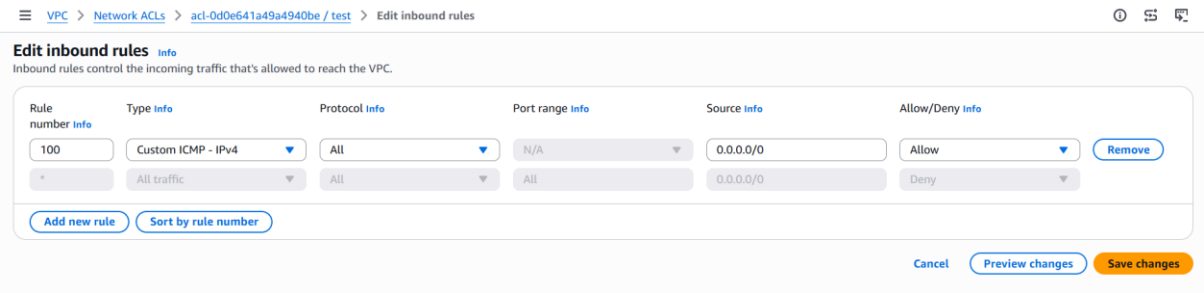


Open the **Subnet Associations** tab of the newly created Network ACL and click **Edit Subnet Associations**. Select the subnet containing the EC2 instances and save the changes so that the subnet starts using the new Network ACL.

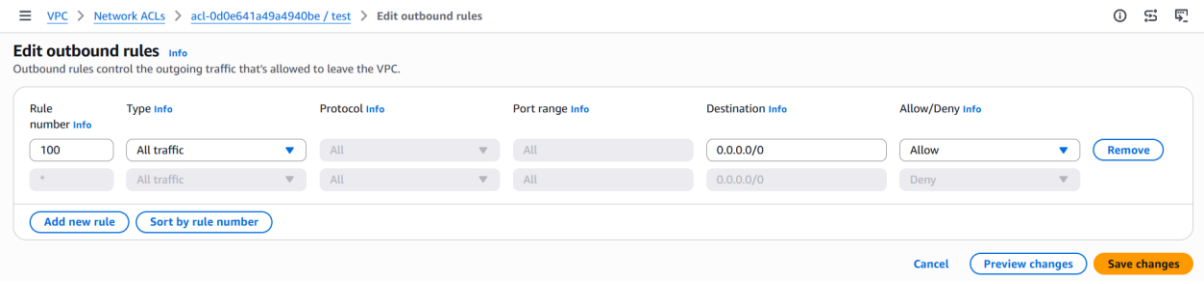



After associating the subnet with the new Network ACL, verify that you can no longer access the EC2 instances using **SSH**, **Ping**, or **HTTP**. This happens because a newly created Network ACL denies all inbound and outbound traffic by default.


Open the **Inbound Rules** section of the Network ACL and click **Edit Inbound Rules**. Add a new rule with **Rule Number 100**, allow **ICMP (Ping)** traffic from **0.0.0.0/0**, and save the changes.



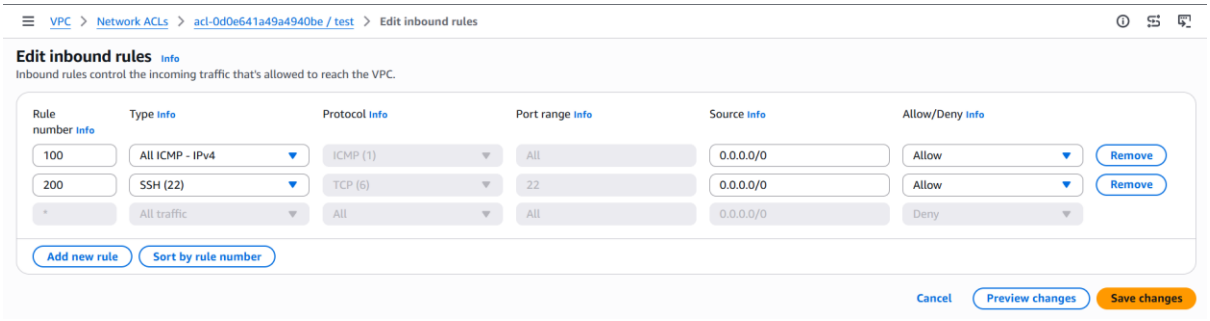

Open the **Outbound Rules** section and edit the rules by adding **Rule Number 100** that allows **All Traffic** to **0.0.0.0/0**. Since Network ACLs are **stateless**, both inbound and outbound rules must be configured for traffic to work correctly.




Test the connection by pinging the EC2 instance again. The ping request should now succeed because ICMP traffic is allowed in the inbound direction and all outbound responses are permitted.

```
amanr@LAPTOP-7OU8O5T1 MINGW64 ~
$ ping 107.23.121.122

Pinging 107.23.121.122 with 32 bytes of data:
Reply from 107.23.121.122: bytes=32 time=211ms TTL=114
Reply from 107.23.121.122: bytes=32 time=213ms TTL=114
Reply from 107.23.121.122: bytes=32 time=211ms TTL=114
Reply from 107.23.121.122: bytes=32 time=211ms TTL=114

Ping statistics for 107.23.121.122:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 211ms, Maximum = 213ms, Average = 211ms

amanr@LAPTOP-7OU8O5T1 MINGW64 ~
$ ping 98.88.70.184

Pinging 98.88.70.184 with 32 bytes of data:
Reply from 98.88.70.184: bytes=32 time=213ms TTL=114
Reply from 98.88.70.184: bytes=32 time=215ms TTL=114
Reply from 98.88.70.184: bytes=32 time=213ms TTL=114
Reply from 98.88.70.184: bytes=32 time=214ms TTL=114

Ping statistics for 98.88.70.184:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 213ms, Maximum = 215ms, Average = 213ms

amanr@LAPTOP-7OU8O5T1 MINGW64 ~
```

Return to the **Inbound Rules** and add another rule with **Rule Number 200** to allow **SSH (TCP Port 22)** from **0.0.0.0/0**. Save the changes and verify that you can now establish an SSH connection to the EC2 instance.




Now you can see we are able to connect to the instance